



中山大學
SUN YAT-SEN UNIVERSITY

指针与数组进阶与练习

學大山中立國
中山大学计算机学院



主讲人: 陈武辉

目录

CONTENTS

01

合并两个有序数组

02

选择排序

03

冒泡排序

04

求第n小数

05

$O(n)$ 的含义

合并两个有序数组

题目描述：给你两个升序整数数组 `nums1` 和 `nums2`，另有两个整数 `m` 和 `n`，分别表示 `nums1` 和 `nums2` 中的元素数目。已知 `nums1` 有 `m+n` 个元素，请你合并 `nums2` 到 `nums1` 中，并使得 `num1` 升序排列。

示例1：

输入： `nums1 = [1,2,3,0,0,0]`, `m = 3`,

`nums2 = [2,5,6]`, `n = 3`

输出： `[1,2,2,3,5,6]`

示例2：

输入： `nums1 = [1]`, `m = 1`, `nums2`

`= []`, `n = 0`

输出： `[1]`

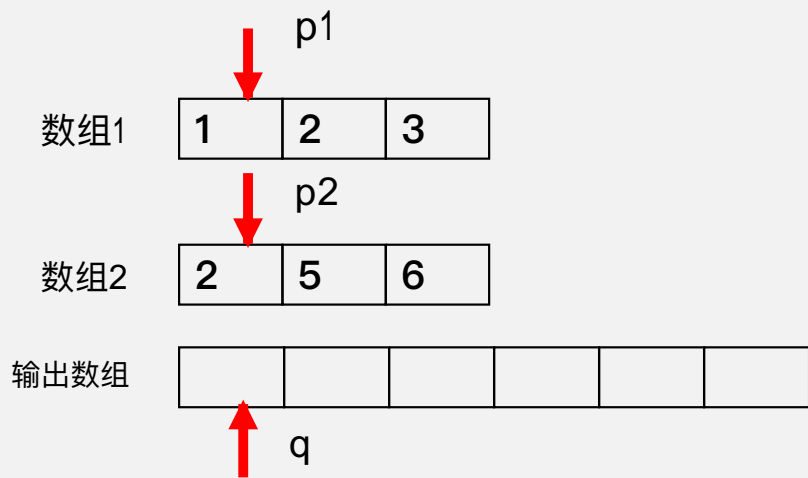
进阶：

- 你可以设计实现一个时间复杂度为 $O(m + n)$ 的算法解决此问题吗？

题目链接： [88. 合并两个有序数组 - 力扣 \[Leetcode\]](#)

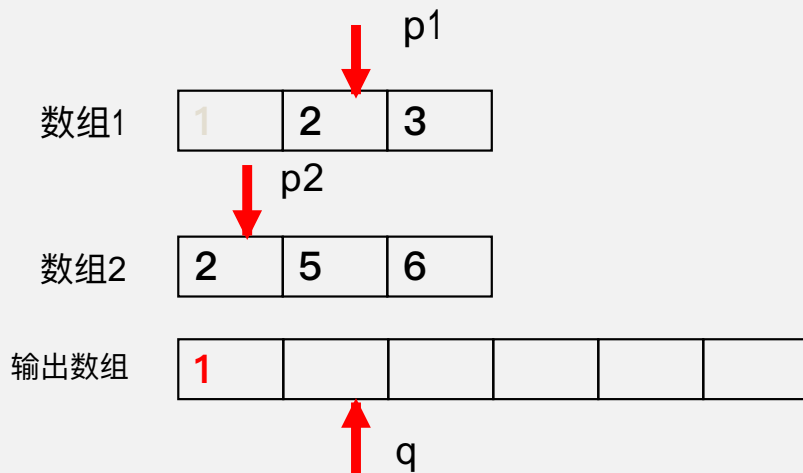
合并两个有序数组

方法1: 开辟一个m+n的临时空间, 总是将最小的数写入新数组, 直到写完。



第0步: 初始化指针指向

- p1, p2 指向两个数组待插入元素
- q 指向添加位置

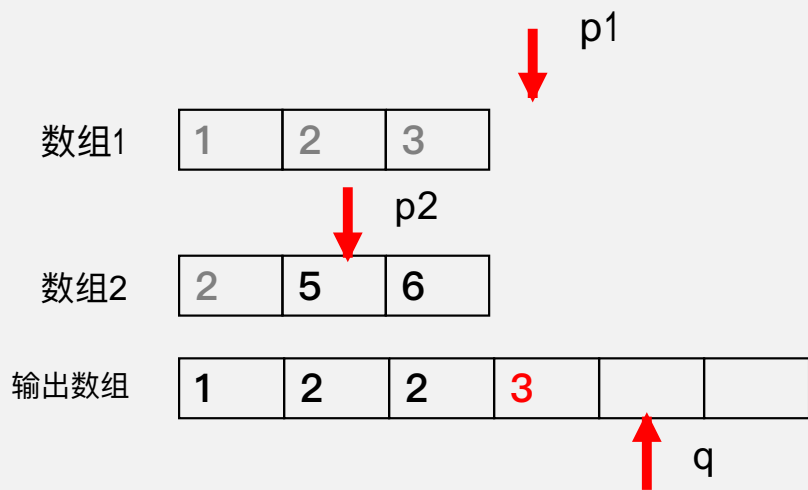


第1步: 将较小的数 1 写入q

- p1++, q++。
- 指针含义不变

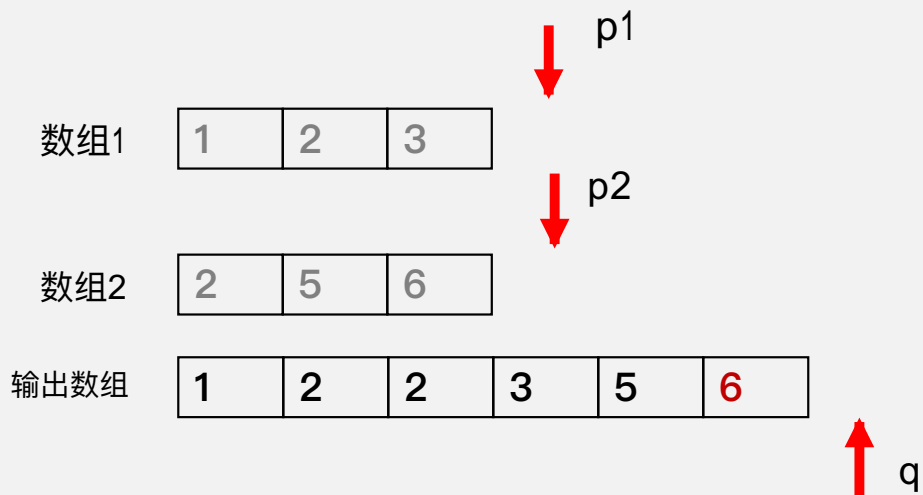
合并两个有序数组

方法1: 开辟一个m+n的临时空间, 总是将最小的数写入新数组, 直到写完。



第4步: p1已没有元素

- $p1 == m$, p2 指向待插入元素
- q 指向添加位置



第6步: 将剩余数字写入 q

- $p1 == m \ \&\& \ p2 == n$ 任务完成
- q 指向添加位置

合并两个有序数组

方法1: 开辟一个m+n的临时空间, 总是将最小的数写入新数组, 直到写完。

```
void merge1(int *nums1, int m, int *nums2, int n) {
    int *p1 = nums1, *p1e = nums1 + m;
    int *p2 = nums2, *p2e = nums2 + n;
    int new[m+n];
    int *q = new;

    while (p1<p1e || p2<p2e) {
        if (p1<p1e && p2<p2e)
            if (*p1<=*p2) *(q++)=*(p1++);
            else *(q++)=*(p2++);
        else if (p1<p1e)
            *(q++)=*(p1++);
        else if (p2<p2e)
            *(q++)=*(p2++);
    }

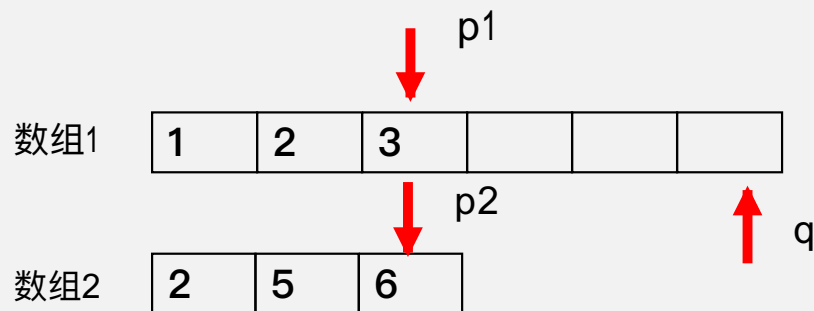
    for (int *p=new;p<q;p++,nums1++)
        *nums1 = *p;
}
```

要点:

1. 程序严格分为三个段落
 - 初始化
 - 数据处理
 - 数组复制
2. 初始化
 - 使用结束指针, 如: p1e
3. 数据处理
 - 分析结束条件
 - if else if
 - 多路选择, 思考如何从图示得到选择结构
4. 数组复制
 - 指针直接取数, 不需要做索引加法, 如 arr+i 再取数, 效率高!

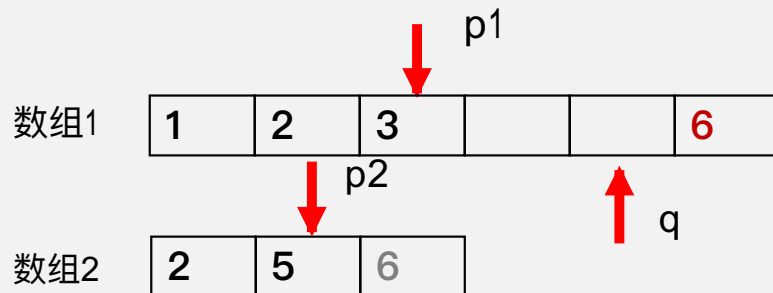
合并两个有序数组

方法2：根据题意，可利用 num1 的空间完成数组合并，如图所示。



第0步：初始化指针指向

- p1, p2 指向两个数组待插入元素
- q 指向添加位置
- 如果 $q == p1$ 则表示插完了

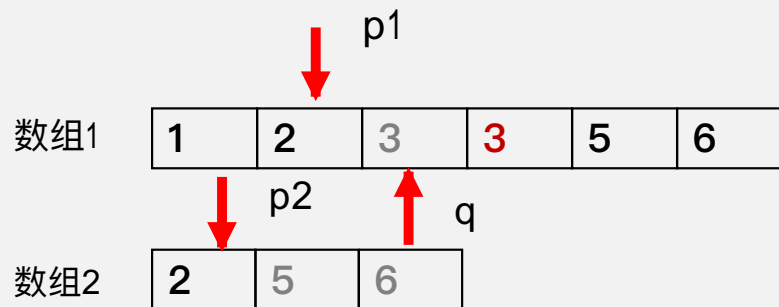


第1步：将较大的数 6 写入 *q

- p2--, q--。
- 指针含义不变

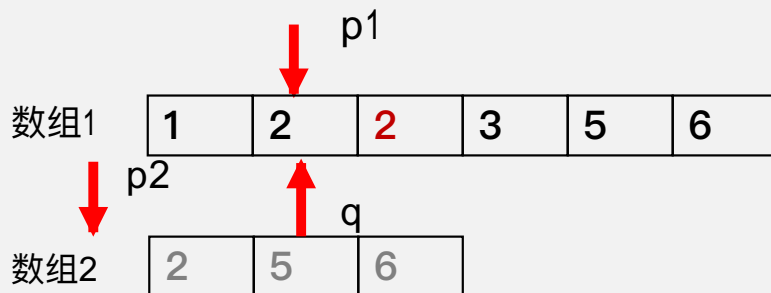
合并两个有序数组

方法2：根据题意，可利用 num1 的空间完成数组合并，如图所示。



第3步：p1 移动情况

- p1--, q--
- 指针含义不变



第4步：*p2 >= *p1, 则 *q = *p2--

- q--。
- 结束条件 p1 >= q。也可以是 p2 < nums2

合并两个有序数组

方法2：根据题意，可利用 num1 的空间完成数组合并，如图所示。

```
void merge(int *nums1, int m, int *nums2, int n) {
    int *p1 = nums1 + m - 1;
    int *p2 = nums2 + n - 1;
    int *q = nums1 + m + n - 1;

    while (p1 < q) {
        if (*p2 >= *p1) *(q--) = *(p2--);
        else *(q--) = *(p1--);
        // printf("%d\n", *(q+1));
    }
}
```

要点：

1. 正确分析循环结束条件
2. 优先搬 nums2，即 $*p2 \geq *p1$
3. 一定要分析 $m=0$ 和 $n=0$

条件下，函数的正确性哦

- $n = 0$ 对吗？
- $m = 0$ 对吗？

如果不对，如何完善呢？



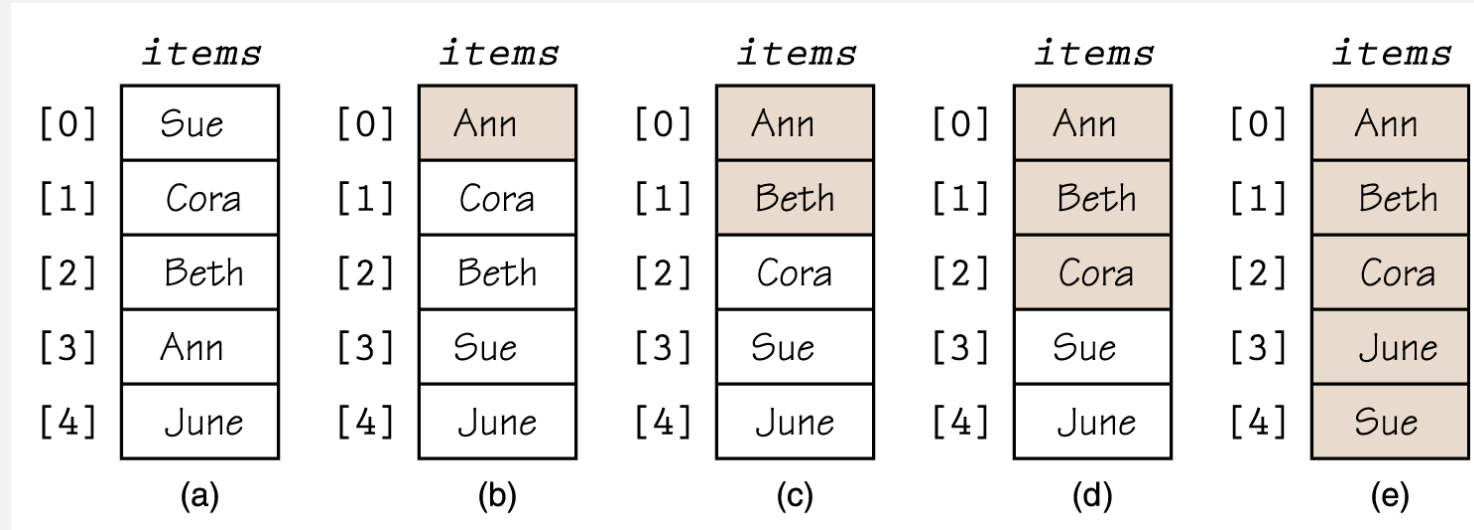
$O(n)$ 是什么意思?

- 一种度量时间或空间消耗的上界的表示
- 例如：合并有序数组
 - 算法二：直接把B数组合并到A数组，则最多需要 $n+m$ 次赋值
 - 算法一：采用一个中间数组合并，再复制到数组A，则最多需要 $2(n+m)$ 次赋值
 - 因此，合并有序数组的计算时间大致与 $n+m$ 线性相关，即
 - 存在数 a, b 使得计算时间 $T < a * (m+n) + b$
 - 则时间上界 $O(m+n)$
 - 其中, n, m
是数组的大小，即问题的规模。即计算时间和问题规模线性相关。即规模越大，耗时越多。问题规模大一倍，耗时大约多一倍。
 - 尽管算法二可能比算法一快一倍以上，这非常有意义。但从计算理论的角度，算法没有本质变化

Sorting

- Because sorting a large number of elements can be **extremely time-consuming**, a good sorting algorithm is **very desirable**
- We present several quite different sorting algorithms

Selection Sort (图示法)



Example of a selection sort (sorted elements are shaded)

Select Sort (伪代码表示法)

```
list = {Sue, Cora, Beth, Ann, June}  
FOR i from 0 to list.length-2 DO  
    Find index of the smallest item in list[i..list.length-1]  
    IF index <> i THEN  
        list.swap (i, index)  
    END IF  
END FOR
```



```
0, 1, 2, 3, 4  
list = {Sue, Cora, Beth, Ann, June}  
FOR i from 0 to list.length-2 DO  
    index = i  
    FOR j from i+1 to list.length-1 DO  
        IF list[j] < list[index] THEN index=j  
    END FOR  
    ...  
END FOR
```

选择排序 (Selection Sort)

根据前述位代码，我们直接翻译成 C 语言

```
void SelectedSortInt(int *nums, size_t n) {  
    for (int i=0; i<=n-2; i++) {  
        //find minimums idx  
        int idx=i;  
        for (int j=i+1; j<=n-1; j++)  
            if (nums[j]<nums[idx])  
                idx = j;  
        //swap  
        if (i!=idx) {  
            int temp = nums[i];  
            nums[i] = nums[idx];  
            nums[idx] = temp;  
        }  
    }  
}
```

Bubble Sort (图示法)

<i>items</i>		<i>items</i>		<i>items</i>		<i>items</i>		<i>items</i>	
[0]	Phil	[0]	Phil	[0]	Phil	[0]	Phil	[0]	A1
[1]	A1	[1]	A1	[1]	A1	[1]	A1	[1]	Phil
[2]	John	[2]	John	[2]	Bob	[2]	Bob	[2]	Bob
[3]	Jim	[3]	Bob	[3]	John	[3]	John	[3]	John
[4]	Bob	[4]	Jim	[4]	Jim	[4]	Jim	[4]	Jim

a) First iteration (Sorted elements are shaded.)

<i>items</i>		<i>items</i>		<i>items</i>		<i>items</i>	
[0]	A1	[0]	A1	[0]	A1	[0]	A1
[1]	Phil	[1]	Bob	[1]	Bob	[1]	Bob
[2]	Bob	[2]	Phil	[2]	Jim	[2]	Jim
[3]	John	[3]	Jim	[3]	Phil	[3]	John
[4]	Jim	[4]	John	[4]	John	[4]	Phil

b) Remaining iterations (Sorted elements are shaded.)

Figure 9.10 Example of a bubble sort

Bubble Sort (完善算法并用C语言实现)

```
list = {Sue, Cora, Beth, Ann, June}  
FOR i from 0 to list.length-2 DO  
    从 list.length-1 到 i+1,  
    分别与前面一个元素比较, 将小的元素交换到前面  
END FOR
```

实现的函数原型如下:

```
void bubbleSortInt(int *base, size_t n);
```

某同学发现, 如果在某轮冒泡中, 如果没有一次交换, 则排序已完成! 请问如何修改程序?



求一个数组中第 k 小元素

给定 n 个数元素的整数数组，请计算第 k 小元素，其中 $k \ll \log_2 n$ 。

(注，这时使用 qsort 不划算，冒泡排序的原理最合适)

```
] int RankingK(int *nums, size_t n, int k) {  
  ]   for (int i=0; i<=n-2; i++) {  
  ]       for (int j=n-1; j>=i+1; j--) {  
  ]           if (nums[j]<nums[j-1]) {  
  ]               int temp = nums[j];  
  ]               nums[j] = nums[j-1];  
  ]               nums[j-1] = temp;  
  ]           }  
  ]       }  
  ]       if(k==i+1)  
  ]           return nums[i];  
  ] }
```



算一算 $O(n)$

- 排序算法我们最关注数组上两个数比较的次数
- 已选择排序为例，数组长度为 n ，需要多少次比较？



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

學大山中立國

中山大学计算机学院



编制人: 课题组